Módulo: Documentación de experimentos (Reproducibilidad y Replicabilidad)

Curso Experimentación Computacional

1. Objetivo general

Este módulo busca que los estudiantes apliquen de manera práctica los principios de reproducibilidad y replicabilidad científica, siguiendo las guías de *ACM Artifact Review & Badging*[1]. El trabajo se organizará en torno a un proyecto experimental en el que se implementará, documentará, ejecutará y analizará un benchmark comparativo.

2. Proyecto experimental del módulo

El módulo se desarrollará en torno al proyecto *"Benchmark reproducible de multiplicación de matrices NxN en C++, Go, Java y Python"*.

El objetivo del proyecto será diseñar y desarrollar un experimento computacional reproducible que permita comparar el tiempo de ejecución del algoritmo naive[2] de multiplicación de matrices cuadradas NxN implementado en C++, Go, Java y Python.

La pregunta de investigación será:

¿Existen diferencias estadísticamente significativas en el tiempo de ejecución del algoritmo naive de multiplicación de matrices NxN entre implementaciones equivalentes desarrolladas en C++, Go, Java y Python, y cómo varían estas diferencias según el tamaño de la matriz?

Se plantearán las siguientes hipótesis:

- H_0: El lenguaje de implementación no produce diferencias estadísticamente significativas en el tiempo de ejecución del algoritmo de multiplicación de matrices.
- H_1: Al menos uno de los lenguajes presenta un tiempo de ejecución significativamente diferente de los demás.

[1] https://www.acm.org/publications/policies/artifact-review-and-badging-current?utm_source=chatgpt.com
[2] Algoritmo naive: versión más directa y sencilla de resolver un problema, sin aplicar optimizaciones especiales.

3. Contenido

Semana 1 – Diseño experimental, GitHub y documentación del artifact

Se deberá crear un repositorio para el proyecto en GitHub, el cual será utilizado como espacio de desarrollo, documentación y control de versiones del artifact durante todo el módulo.

En este repositorio deberán elaborar una versión inicial del archivo *README.md* (formato Markdown), que servirá como documento base del proyecto experimental. Este documento deberá incluir el objetivo y la pregunta de investigación, las hipótesis nula y alternativa, las variables independientes y dependientes, y el diseño experimental.

Durante la clase, cada estudiante dispondrá de diez minutos para presentar y discutir su propuesta experimental. Además, dará sus comentarios respecto de los principios y criterios de *ACM Artifact Review & Badging* como marco de referencia para la construcción, documentación y posterior evaluación del artifact científico desarrollado durante este módulo de la asignatura.

Semana 2 – Implementación y ejecución del experimento

Se deberá implementar el algoritmo naive de multiplicación de matrices NxN en C++, Go, Java y Python. Las cuatro implementaciones deberán ser funcionalmente equivalentes, utilizando el mismo algoritmo, tipo de datos, matrices de entrada y criterio para medir el tiempo de ejecución. No se podrán utilizar librerías optimizadas para multiplicación de matrices.

Las matrices deberán generarse de manera controlada, utilizando una semilla previamente definida que permita reproducir los mismos datos de entrada. El experimento se ejecutará para matrices de tamaños N = 500, N=1000 y N=1500, realizando 10 repeticiones por cada combinación de lenguaje y tamaño de matriz.

La medición del tiempo de ejecución deberá considerar exclusivamente la operación de multiplicación de matrices. El tiempo utilizado para generar, inicializar o cargar las matrices de entrada no deberá incluirse en la medición. Las matrices deberán estar completamente creadas antes de iniciar el temporizador; la medición comenzará inmediatamente antes de ejecutar el algoritmo de multiplicación y finalizará una vez obtenida la matriz resultante.

Los resultados obtenidos por cada implementación deberán almacenarse en cuatro archivos independientes, uno por lenguaje: *cpp.csv*, *go.csv*, *python.csv* y *java.csv*. Los cuatro archivos deberán utilizar una misma estructura tabular que contenga las columnas *language*, *N*, *repetition* y *time_seconds*. Donde *language* identifica el lenguaje utilizado, *N* corresponde al tamaño de la matriz, *repetition* indica el número de la ejecución y *time_seconds* registra el tiempo empleado en la multiplicación de las matrices.

Se deberán además registrar las principales características del entorno de ejecución, incluyendo procesador, memoria RAM, sistema operativo, arquitectura y versiones de los lenguajes, compiladores o máquinas virtuales utilizadas.

Durante la clase, cada estudiante presentará sus implementaciones y los primeros resultados obtenidos, justificando las decisiones adoptadas para asegurar condiciones de ejecución comparables entre los cuatro lenguajes.

Semana 3 – Entorno reproducible y evaluación funcional del artifact

Se deberán preparar entornos reproducibles utilizando *Docker*, mediante cuatro imágenes correspondientes a las implementaciones en C++, Go, Python y Java. Cada imagen deberá incluir las herramientas necesarias para compilar y/o ejecutar la implementación correspondiente. La construcción de estas imágenes Docker deberá realizarse manualmente.

El archivo *README.md* deberá actualizarse con instrucciones claras y completas para que un tercero pueda ejecutar el experimento hasta este punto. La ejecución de cada implementación deberá generar un archivo de resultados independiente, correspondiente a cada lenguaje: *cpp.csv*, *go.csv*, *python.csv* y *java.csv*. Estos archivos deberán utilizar una estructura común de columnas (*language*, *N*, *repetition* y *time_seconds*) que permita su posterior integración y análisis.

Durante esta semana se utilizarán los criterios de *ACM Artifact Review & Badging* para revisar si el artifact se encuentra correctamente documentado, es ejecutable y permite verificar que sus componentes funcionan de acuerdo con lo descrito. El objetivo será que el proyecto alcance las características esperadas de un *artifact Functional*.

Durante la clase, cada estudiante realizará una breve demostración de la ejecución de su experimento utilizando los contenedores Docker definidos, verificando la generación de los cuatro archivos de resultados, y presentará la documentación actualizada de su artifact.

Semana 4 – Automatización y análisis estadístico

Se deberá construir un pipeline reproducible utilizando *Snakemake*, de manera que las principales etapas del experimento puedan ejecutarse de forma automatizada y con mínima intervención manual. El pipeline deberá coordinar la creación y ejecución de los contenedores Docker correspondientes a las implementaciones en C++, Go, Python y Java.

Las imágenes Docker utilizadas por el pipeline deberán construirse previamente en forma manual a partir de los archivos Dockerfile incluidos en el artifact. No se permitirá utilizar imágenes previamente construidas y publicadas en repositorios de imágenes (Docker Hub, GitHub Container Registry u otros equivalentes).

El pipeline deberá permitir:

- Ejecutar los cuatro contenedores Docker correspondientes a C++, Go, Python y Java.
- Ejecutar el experimento para matrices de tamaños N = 500, N = 1000 y N = 1500.
- Realizar 10 repeticiones por cada combinación de lenguaje y tamaño de matriz.
- Generar los archivos de resultados *cpp.csv*, *go.csv*, *python.csv* y *java.csv*.
- Integrar automáticamente estos cuatro archivos en un único archivo *benchmark.csv*.
- Ejecutar automáticamente el análisis estadístico a partir de *benchmark.csv*.
- Generar las tablas y gráficos correspondientes.

El análisis estadístico deberá considerar como factores el lenguaje de programación y el tamaño de la matriz, permitiendo analizar tanto sus efectos individuales como la posible interacción entre ambos. Se utilizará ANOVA de dos factores con interacción y, cuando corresponda, comparaciones post-hoc mediante *Tukey HSD*.

Snakemake deberá definir explícitamente las dependencias entre las distintas etapas del workflow, desde la ejecución de los contenedores Docker y la generación de los archivos de resultados individuales hasta su integración en *benchmark.csv* y la posterior generación del análisis estadístico, tablas y gráficos.

Una vez construidas manualmente las cuatro imágenes Docker, el pipeline deberá permitir reproducir automáticamente el flujo experimental mediante un único comando o una secuencia mínima claramente documentada en el archivo *README.md*.

En esta semana se revisará nuevamente el artifact utilizando los criterios de *ACM Artifact Review & Badging*, poniendo especial atención en su documentación, automatización y facilidad de reutilización. El objetivo será avanzar desde un *artifact funcional* hacia uno con características de *Reusable*.

Durante la clase, cada estudiante realizará una demostración de la ejecución completa del pipeline y presentará la documentación actualizada del artifact.

Semana 5 – Publicación del artifact y evaluación cruzada

Se deberá preparar y publicar una versión estable de su artifact en *Zenodo*, de manera que pueda ser archivada, identificada mediante un DOI y utilizada posteriormente durante la evaluación cruzada.

La versión publicada deberá incluir, como mínimo:

- Código fuente de las implementaciones en C++, Go, Java y Python.
- Archivo *README.md* con las instrucciones completas de reproducción.
- Archivos *Dockerfile* y demás archivos necesarios para construir los entornos reproducibles correspondientes a las cuatro implementaciones.
- Pipeline de automatización definido mediante *Snakefile*.
- Archivos de resultados *cpp.csv*, *go.csv*, *python.csv* y *java.csv*.

- Archivo consolidado *benchmark.csv*.
- Resultados del análisis estadístico.
- Tablas y gráficos generados.
- Licencia de uso.
- Mini datasheet FAIR que describa los datos generados, su procedencia, estructura, limitaciones y condiciones de reutilización.

Una vez publicado el artifact, se deberá obtener el DOI proporcionado por Zenodo e incorporarlo en el archivo *README.md* del repositorio en GitHub.

Posteriormente, cada estudiante deberá evaluar el artifact publicado por un compañero mediante una revisión cruzada inspirada en el proceso de *ACM Artifact Review & Badging*. El evaluador deberá intentar reproducir el experimento utilizando únicamente el artifact publicado, su archivo *README.md* y los archivos de configuración y automatización incluidos. Para ello, deberá construir los entornos *Docker* correspondientes y ejecutar el pipeline definido mediante *Snakemake*, verificando que sea posible reproducir el flujo completo desde las implementaciones hasta la generación de los resultados, análisis estadístico, tablas y gráficos.

Durante esta evaluación se deberán registrar las dificultades encontradas, dependencias faltantes, errores o ambigüedades en la documentación, modificaciones necesarias y posibles diferencias entre los resultados obtenidos y los reportados originalmente. También se deberá verificar que los cuatro contenedores Docker puedan ejecutarse correctamente, que se generen los archivos cpp.csv, go.csv, python.csv y java.csv, y que estos sean integrados automáticamente en el archivo *benchmark.csv*.

Durante la última clase, cada estudiante presentará los principales resultados de su experimento, la versión publicada de su artifact en *Zenodo* y la experiencia de evaluación cruzada.

3. Evaluación

- En cada semana los estudiantes obtienen una nota por su trabajo.
- Nota Final = Promedio de las cinco notas obtenidas en las semanas.

4. Bibliografía

- ACM Artifact Review and Badging https://www.acm.org/publications/policies/artifact-review-and-badging-current?utm_source=chatgpt.com
- Reproducibility and Replicability in Science https://nap.nationalacademies.org/catalog/25303/reproducibility-and-replicability-in-science

- Best Practices for Replicability, Reproducibility and Reusability of Computer-Based Experiments Exemplified by Model Reduction Software
  https://arxiv.org/pdf/1607.01191.pdf